

CoderCommands MCP System

Step 08 Record — Local Verification

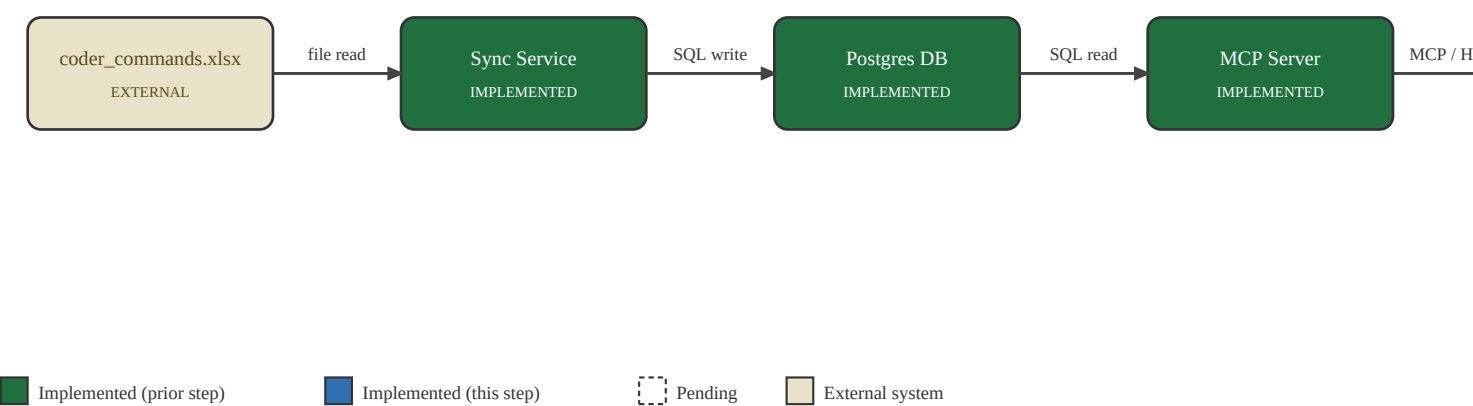
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	08 — Local Verification
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Run a dedicated, broader verification pass against the real running compose stack -- not individual components in isolation. Two things hadn't yet been proven against the actual deployment: that a live xlsx edit reaches Postgres with zero manual steps, and that search quality holds up across a wider, less cherry-picked set of natural-language questions.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	(no new files)	This step is verification only -- exercises the stack built in Steps 1–7 against the real docker-compose deployment.	—	—

4. Work Performed

- Confirmed all four containers reach a running/healthy state with correct startup order via docker compose ps.
- Live-edit test: temporarily pointed the sync service at a scratch copy of the workbook (never the real coder_commands.xlsx), appended a new row to the tmux sheet while the container was already running and unattended, and waited one poll cycle with no manual trigger of any kind.
- Restored the sync service to the real workbook afterward and confirmed it re-synced back to the full 608-row dataset.
- Ran 6 natural-language queries plus a browse and a list_topics call through the live gateway at 127.0.0.1:8100, deliberately including questions not used to develop or debug the search fix in Step 5, to get an honest read on search quality rather than re-confirming already-tuned cases.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
"remove a docker image" returned 'docker original:orTag original:newTag' (an image tagging/alias command) as the top result instead of any actual deletion command.	Checked the full IMAGE MANAGEMENT section directly: the source spreadsheet has no row for removing a single named image (no 'docker rmi imgName' or 'docker image rm imgName') -- only 'docker image prune' (dangling images) and 'docker system prune -a' (unused + dangling). The alias row's extra text ('when tagged image is removed, only tag is removed') repeats 'remove' twice, which is enough lexical density to outrank the prune commands even though it isn't the right answer either.	This is a content gap in the reference data, not a defect in ingest, sync, or search -- nothing here to fix in code. Flagged directly to the user: adding a 'docker rmi imgName' row to the docker sheet would close this gap and sync in automatically on the next save.
"copy a file to a remote server over ssh" returned the Dockerfile COPY instruction instead of linux's 'scp ...' row, even though the correct row exists in the data.	ts_rank is purely lexical: both rows match 'copy' and roughly one more term ('file' for COPY's description; 'remote' for scp's), so they're close to tied on raw term coverage. Nothing in the ranking distinguishes "copying a file within a Dockerfile build context" from "copying a file to a remote host over the network" -- that distinction is semantic, and the OR-based term-matching approach chosen in Step 5 has no semantic signal to draw on.	Recorded as a known accepted limitation of v1's full-text search rather than fixed here -- consistent with the original plan's choice of full-text search over embedding-based semantic search for v1 (see CodeCommands.pdf, decision summary). A future iteration could re-rank with embeddings if this kind of miss turns out to matter in practice.

6. Verification Record

Check	Method	Result
All four containers healthy with correct startup order	<code>docker compose ps</code>	PASS
Live <code>xlsx</code> edit (new row appended while sync was already running, unattended) reaches Postgres within one poll cycle, no manual trigger	Row count 608→609 and a new <code>sync_runs</code> entry appeared ~7–12s after the save, against a scratch copy so the real workbook was never touched	PASS
Sync service correctly restored to the real workbook afterward	Re-synced to 608 rows matching the real <code>coder_commands.xlsx</code>	PASS
Broad natural-language query set: correct top result	4 of 6: <code>insert-a-row</code> , <code>tmux attach</code> , <code>git undo commit</code> , <code>mongo show databases</code>	PASS
Broad natural-language query set: incorrect top result, root-caused	2 of 6: <code>docker image removal</code> (data gap, not a defect) and <code>ssh file copy</code> (ranking limitation, accepted for v1)	DOCUMENTED, not fixed

7. Next Step / Open Dependencies

Step 9 registers this gateway with Claude Code (`claude mcp add`) and confirms the tool actually gets selected and used correctly from natural-language prompts in a real session.